Microservices with Spring Boot

ADVANCED JAVA CONCEPTS AND PERSISTENCE

19 May, 2026

Agenda Overview

01 Problem with Monolithic Design

02 Microservices Architecture

03 Advanced Java Concepts

04 Spring Boot

05 Building the Microservice

06 Adding Persistence

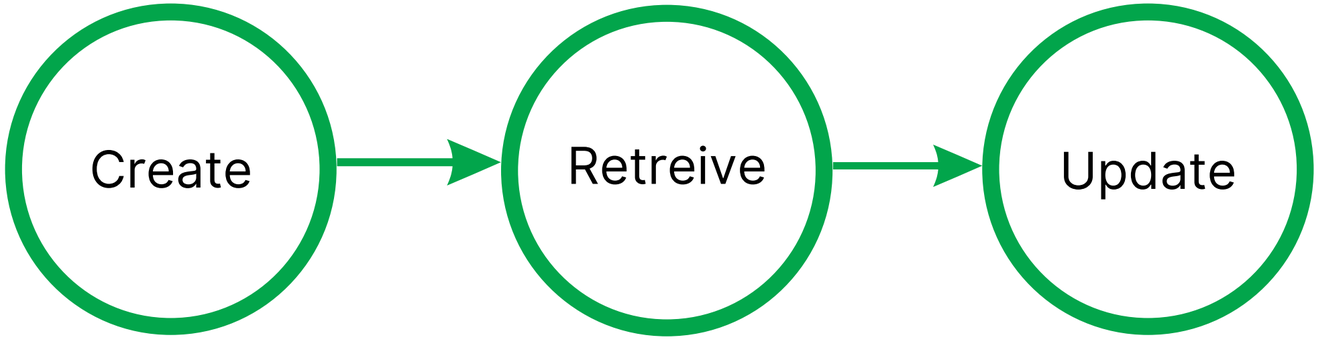
Consider this Scenario:

A hospital's scanning system needs to manage scan requests. Each scan request contains:

- Patient ID
- Scan Type (Chest, Head, Abdomen)
- Scheduled Date
- Status (Pending, Completed, Cancelled)

"How do we build a system that can reliably create, retrieve, update and recover this data?"

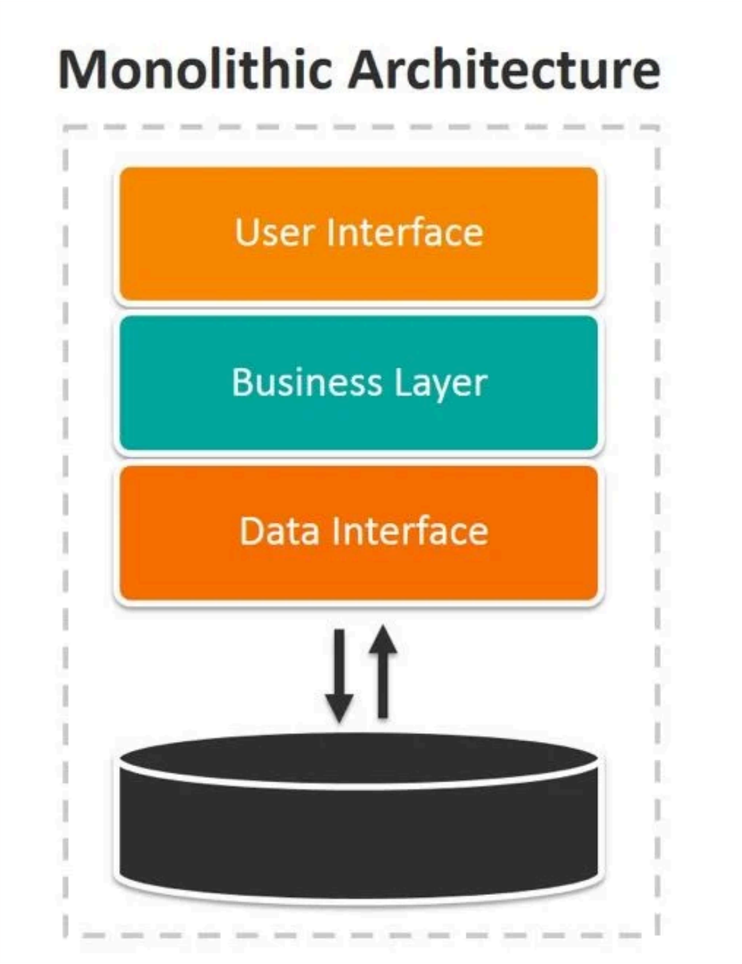
Operations



Scan Request

Scan by ID
Get All Scans

Update Scan Status &
Print Results



What is Monolothic Architecture?

A monolithic application is a single, unified unit where all components of the system: the user interface, business logic, and data access, are tightly bundled together and deployed as one.

In this architecture, every function of the application lives in the same codebase and runs as a single process. When the application needs to handle a request, all layers work together internally without any separation of deployment or responsibility.

This was the standard way of building software for decades, and for small, simple applications it can be straightforward to develop and deploy.

Code – Stage 1

```java
public class ScanSystem {

    //This is how we are storing the data, in memory! When our program stops/restarts all of
    //our data is lost (Problem)
    private static List<String[]> scanRequests = new ArrayList<>();

    //Simple counter to simulate IDs which will reset every run, so can never really store a record (Problem)
    private static int idCounter = 1;
```

Issues with this approach

- All logic exists in a single class (ScanSystem)
- No separation between data, logic, and operations
- Data stored in memory using an ArrayList (lost on restart)
- Application state only exists during runtime
- IDs are managed manually within the same class
- Entire system runs as a single process

Code

Code – Stage 1

```java
/**
 * Prints all scan requests to the console.
 *
 */
public static void getAllScans() {
    if (scanRequests.isEmpty()) {
        System.out.println(x: "No scan requests found.");
        return;
    }

    // Loop through the list and print each scan
    for (String[] scan : scanRequests) {
        printScan(scan);
    }
}
```

No API layer

- Output is written to System.out (console only)
- Data cannot be consumed by other services or frontends
- No standardized interface for communication
- While it is possible to build a web server manually, it requires significant additional effort and complexity

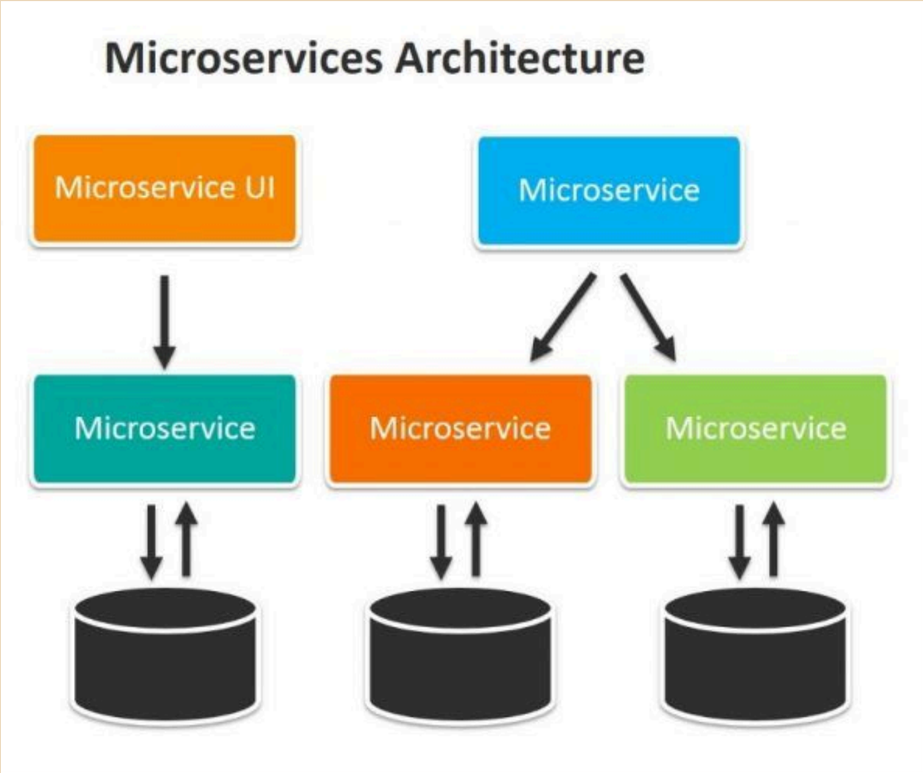
Broader Implications:

- Tightly coupled components
- Difficult to scale independently
- Failures can impact the entire system

Code

Stage 2: Microservices!

A shift toward modular, independently deployable systems designed to improve scalability, maintainability, and reliability.



What are Microservices

A microservices application breaks that single unit into small, independent services with each responsible for one specific job and deployed on its own.
Instead of everything living in one codebase, each service runs as its own process and communicates with others over a network, typically through REST APIs.
This is the modern standard for building scalable software, and it's especially powerful when you need different parts of your system to grow, update, or recover independently of each other.

Before we build...

Understanding the core Java concepts and frameworks behind modern microservices

POJOs

A POJO (Plain Old Java Object)is a simple Java class that represents data. No special rules, no framework dependencies, just fields, constructors, and getters/setters.
It's the cleanest way to model your data in Java. In our case, a scan request is a perfect example of a POJO.

```
List<String[]> scanRequests = new ArrayList<>();

String[] scan = {
    "1", "P001", "Chest CT", "2024-06-01", "PENDING"
};
```

- Clear and structured data representation
- Attributes (fields)
- Easier to read and maintain
- Safer than index-based access
- OOP advantages

```
public class ScanRequest {

    // Private fields - only accessible through getters and setters (Encapsulation)
    private Long id;              // Unique identifier for each scan
    private String patientId;     // The patient this scan belongs to
    private String scanType;      // What kind of scan - Chest, Head etc
    private String scheduledDate; // When the scan is scheduled
    private String status;        // PENDING, COMPLETED, or CANCELLED
```

Attributes & Access Modifiers

An attribute is a variable that belongs to a class and defines the properties of an object. In our particular instance of ScanRequest, each piece of scan data is an attribute (id, patientId, scanType, scheduledDate, status). An advantage of having attributes is that based on how they are declared they give us some control over where the class data can be accessed from and modified.

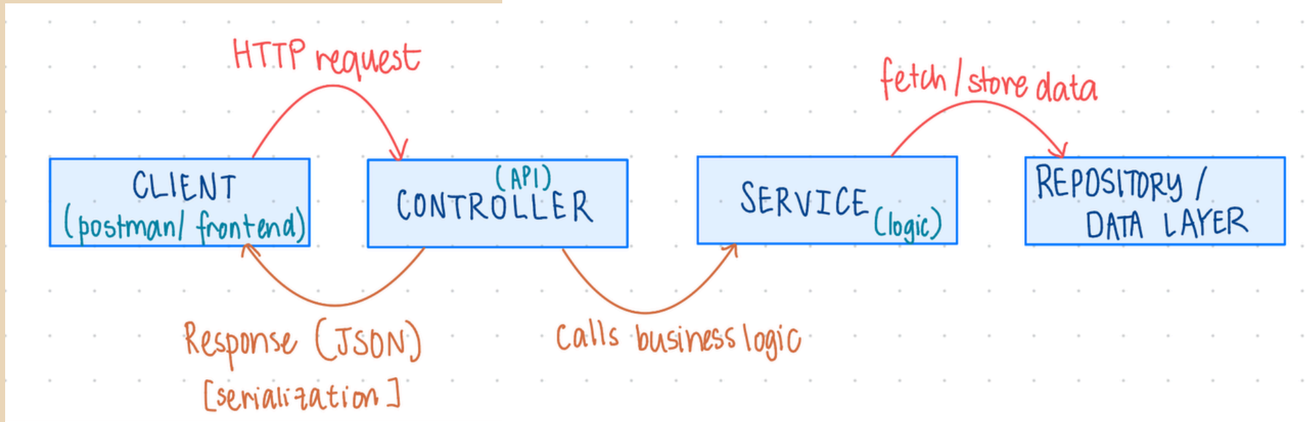
Modifier	Accessible From
Public	Anywhere
Private	Only within the class
Protected	Class and subclasses

```
// PUBLIC getter - controlled READ access to the private attribute
2 usages
public String getPatientId() {
    return patientId;
}
// PUBLIC setter - controlled WRITE access to the private attribute
public void setPatientId(String patientId) {
    this.patientId = patientId;
}
```

SpringBoot

```java
@SpringBootApplication
// boots the entire application

@RestController
// this class handles HTTP requests

@Service
// this class contains business logic

@Autowired
// inject dependencies automatically
```

- Turns Java code into a running web service
- Provides built-in web server (no manual setup)
- Handles HTTP requests and JSON automatically
- Uses annotations to define structure and behavior

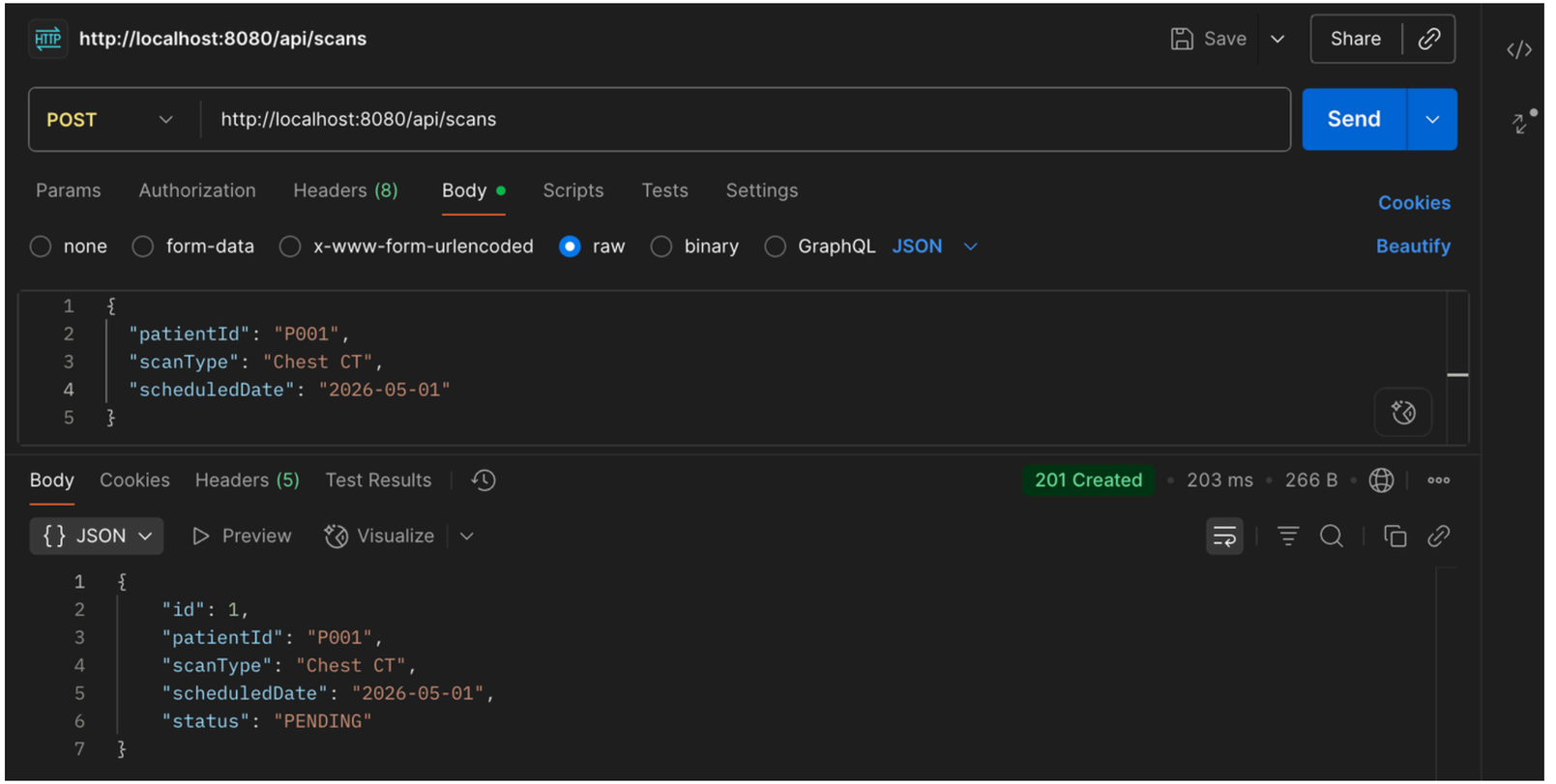


REST – How our Microservices Communicates

Method	Endpoint	What it does
POST	/api/scans	Create a scan
GET	/api/scans	Get all scans
GET	/api/scans/{id}	Get one scan (based on id)
PUT	/api/scans/{id}/status	Update status of one scan
DELETE	/api/scans/{id}	Delete a scan

HTTP Status Codes - How our API communicates back

- 200 OK — request worked, here's your data
- 201 Created — new resource successfully created
- 400 Bad Request — something wrong with what you sent
- 404 Not Found — that resource doesn't exist
- 500 Server Error — something went wrong on our end

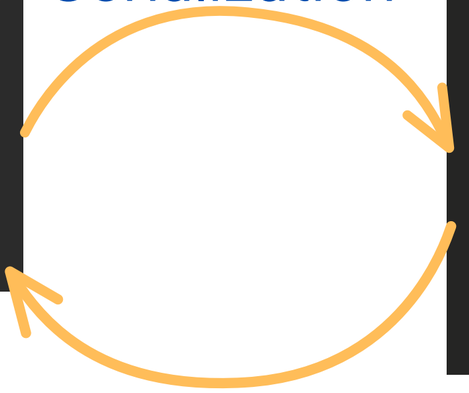


Java <-> JSON

Java Object

```java
ScanRequest scan = new ScanRequest();
scan.setPatientId("P101");
scan.setScanType("CT");
scan.setStatus("PENDING");
```

Serialization



Deserialization

JSON - What travels over the network

```json
{
  "patientId": "P101",
  "scanType": "CT",
  "status": "PENDING"
}
```

Jackson is used internally in Springboot and takes care of this for us!

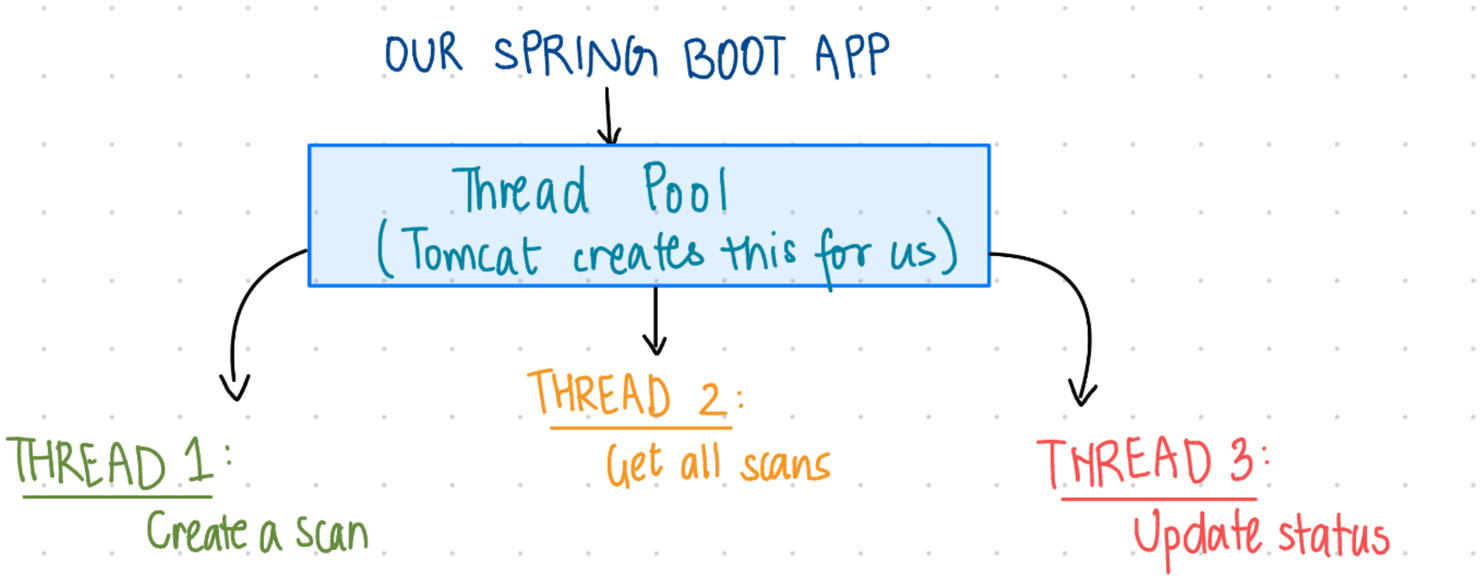
```java
@PostMapping
/**                                          DESERIALIZATION                    */
public ResponseEntity<ScanRequest> createScan(@RequestBody ScanRequest request) {
  ScanRequest created = scanService.createScan(request);
  return ResponseEntity.ok(created);
/**                    SERIALIZATION                                            */
```

Putting it all together!

```java
@RestController                          // handles serialization automatically
@RequestMapping("/api/scans")
public class ScanController {

    @Autowired
    private ScanService scanService; // dependency injection

    @PostMapping                         // POST /api/scans
    public ResponseEntity<ScanRequest> createScan(@RequestBody ScanRequest request) {
        return ResponseEntity.ok(scanService.createScan(request));
    }

    @GetMapping                          // GET /api/scans
    public ResponseEntity<List<ScanRequest>> getAllScans() { ... }

    @GetMapping("/{id}")                 // GET /api/scans/{id}
    public ResponseEntity<ScanRequest> getScanById(@PathVariable Long id) { ... }

    @PutMapping("/{id}/status")      // PUT /api/scans/{id}/status
    public ResponseEntity<ScanRequest> updateStatus(...) { ... }

    @DeleteMapping("/{id}")          // DELETE /api/scans/{id}
    public ResponseEntity<Void> deleteScan(@PathVariable Long id) { ... }
}
```

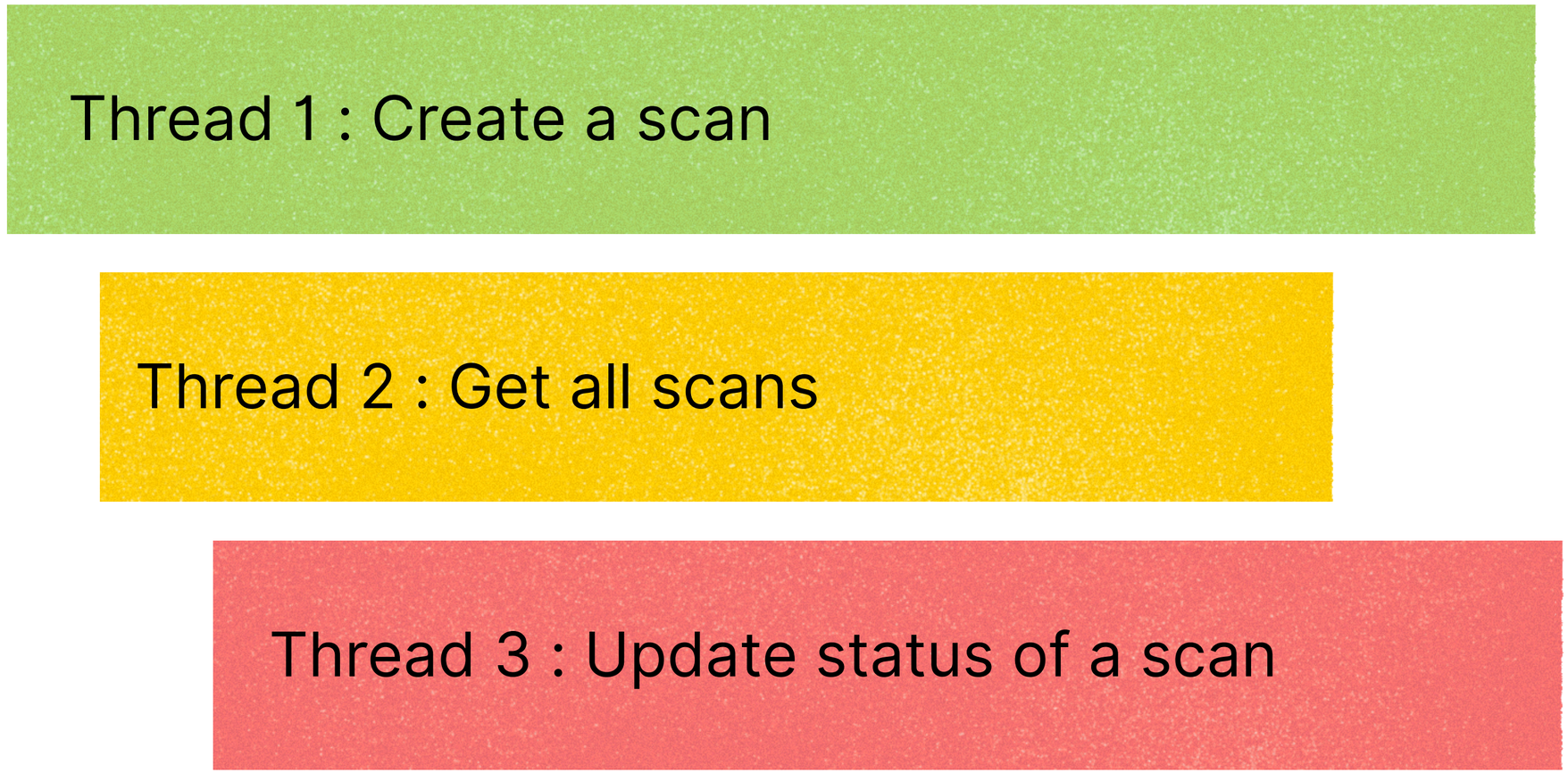
Threads — The Basics

- A thread is a single sequence of instructions being executed
- Your program can run multiple threads at the same time
- Each incoming HTTP request gets its own thread automatically — Spring Boot manages this automatically



Concurrency : Multiple things at once

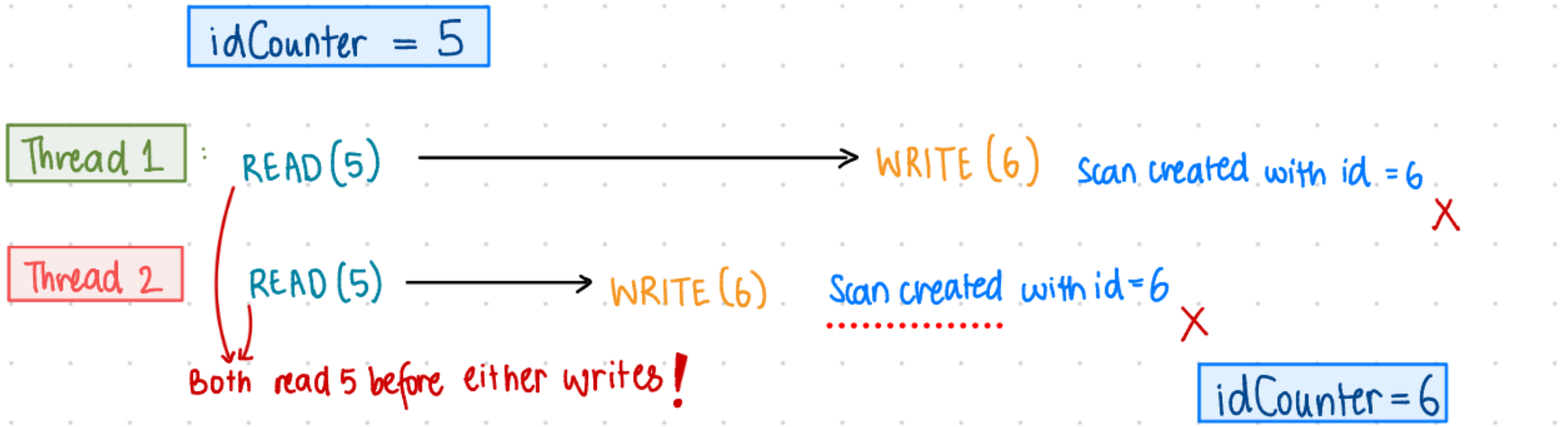
- Concurrency means multiple threads running simultaneously
- Threads share the same memory; this can cause problems
- Shared resources need to be managed carefully



The Problem — Race Conditions

- Consider the scenario where two threads are reading and writing the same variable simultaneously, for example, two threads creating a scan and our code requires each scan to have a unique id.
- Result is unpredictable — data gets corrupted
- Causes duplicate ids, lost records

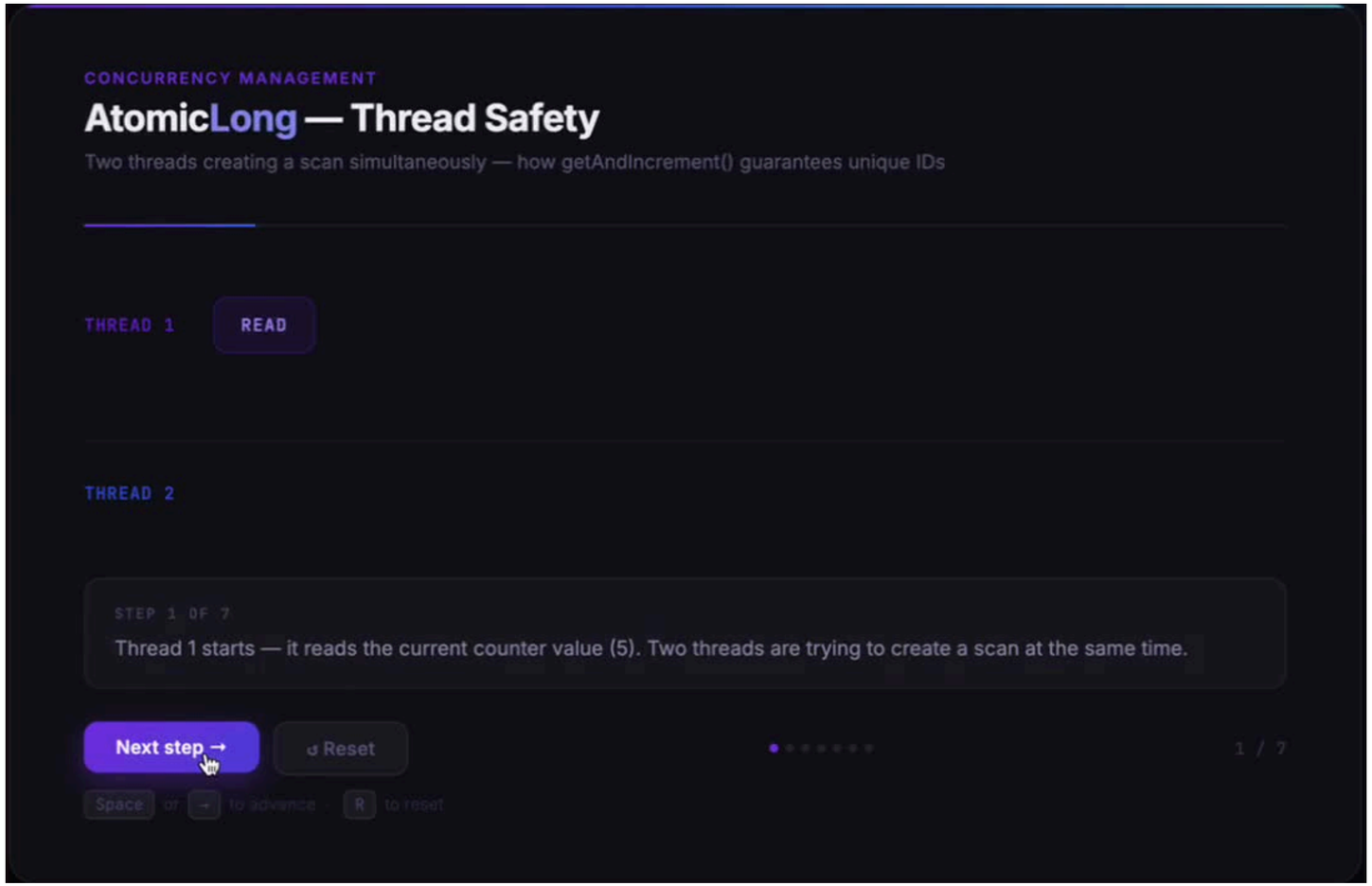
```
//dangerous code - can cause race conditions!
private int idCounter = 5;
request.setId(idCounter++);
```



The Solution — Atomic Long

- AtomicLong makes read-increment-write one uninterruptible step
- Thread 2 cannot jump in between Thread 1's read and write

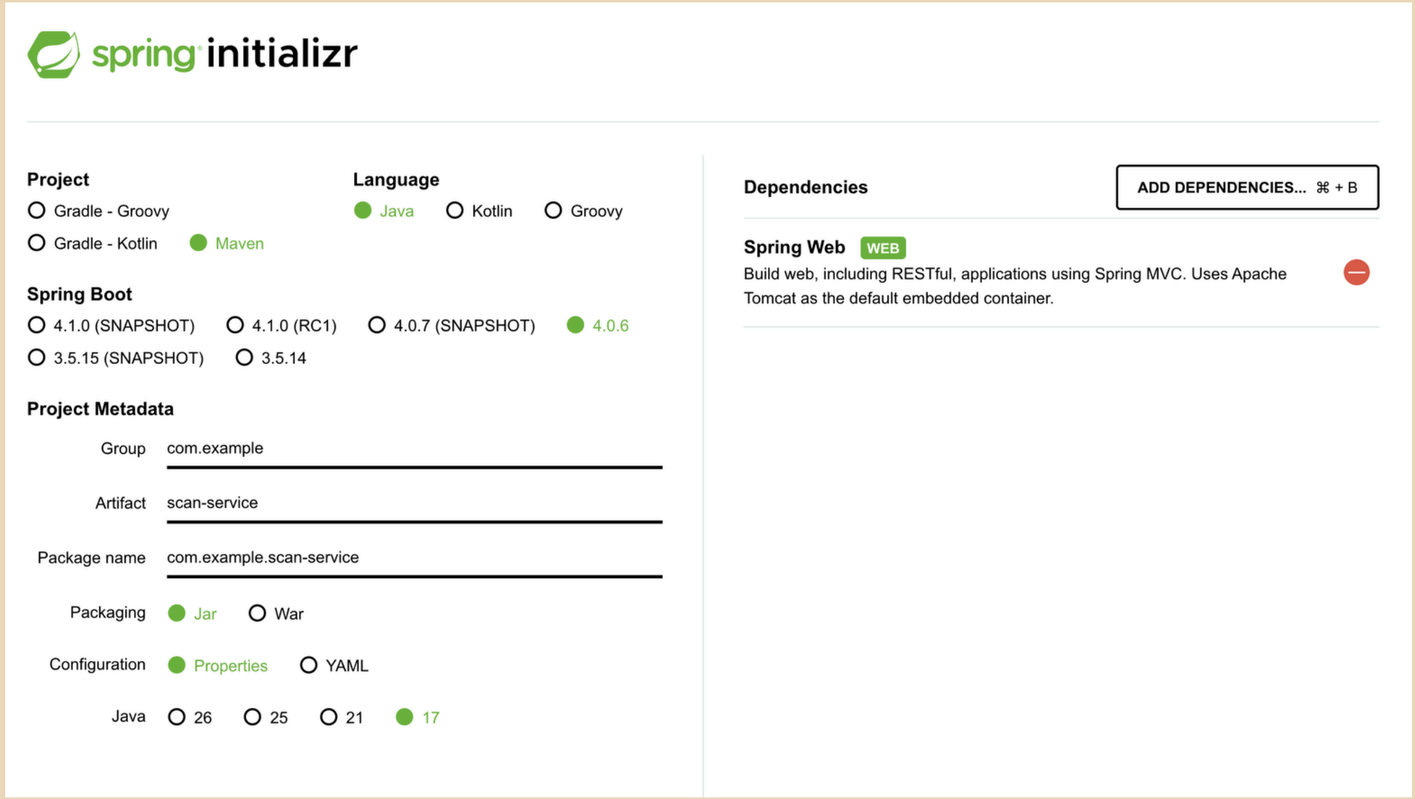
```
// SAFE – AtomicLong is thread safe
private AtomicLong idCounter = new AtomicLong(1);
// getAndIncrement() is ATOMIC –
// one uninterruptible step, no thread can jump in
request.setId(idCounter.getAndIncrement());
```



Every thread is guaranteed a unique value

Code Walkthrough

start.spring.io



Code

Two Problems!

ArrayList is not Thread Safe

Under heavy concurrent load two threads writing simultaneously could corrupt data or lose scan requests entirely

Data is Volatile

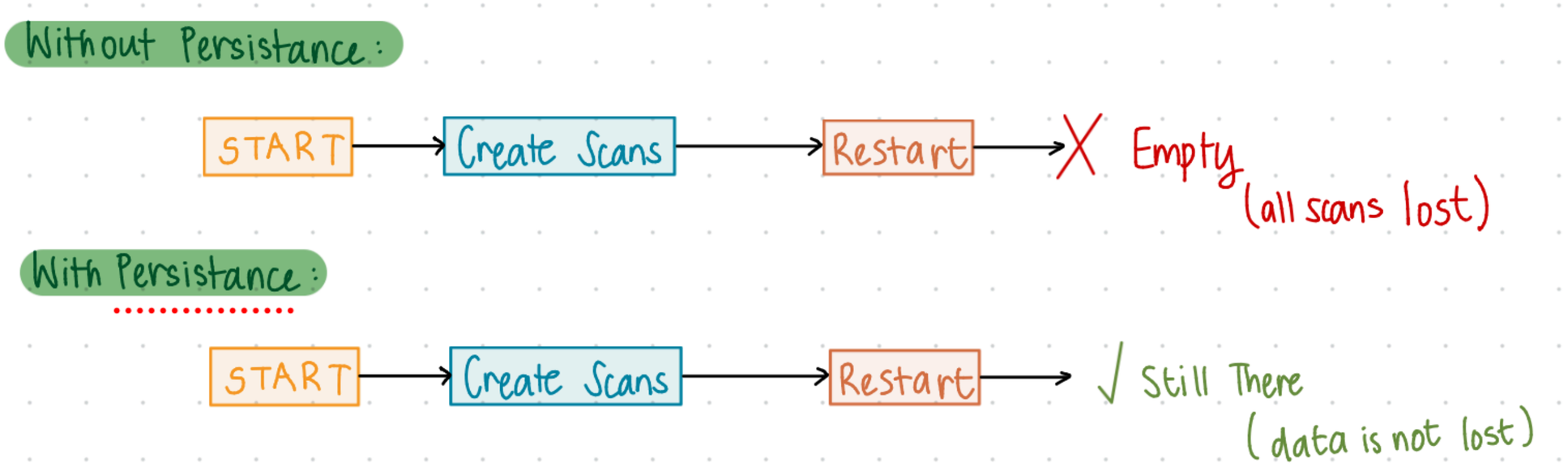
Data doesn’t survive restarts:
If we were to restart the service right now, every scan we just created is gone, No recovery is possible.

Stage 3: Persistance!

Stage 3 solves both problems. We add a database — and suddenly our data survives every restart, and concurrent access is handled safely through ACID transactions.

Persistance — Never Losing Data Again

- Data is written to disk, not memory
- Service can restart or crash, data is always recoverable
- This is exactly the recovery use case we need



Many ways to persist Data

Type	Example	Best for
Embedded DB	H2	Development, testing
Relational DB	PostgreSQL, MySQL	Production Systems
NoSQL	MongoDB	Flexible schemas
In-memory cache	Redis	Speed, temproary data

What Changed

pom.xml

```xml
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>
```

application.properties

```
spring.datasource.url=
  jdbc:postgresql://localhost
  :5432/scandb
spring.jpa.hibernate
  .ddl-auto=update
```

NEW: ScanRepository.java

```java
@Repository
public interface ScanRepository
  extends JpaRepository
  <ScanRequest, Long> {
  // All CRUD for free!
}
```

Controller didn't change. Service barely changed. That's layered architecture.
JPA defines the standard → Hibernate generates the SQL → PostgreSQL stores the data. We write zero SQL

Turning our POJO into a Database Entity

Stage 2:

```
public class ScanRequest {
    private Long id;
    private String patientId;
    private String scanType;
    private String scheduledDate;
    private String status;
}
```

Stage 3:

```
@Entity
@Table(name = "scan_requests")
public class ScanRequest {

    @Id
    @GeneratedValue(
      strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "patient_id",
      nullable = false)
    private String patientId;

    @Column(name = "scan_type",
      nullable = false)
    private String scanType;
}
```

The service layer

Stage 2:

```
// In memory - lost on restart
private List<ScanRequest>
  scanRequests = new ArrayList<>();

// Manual loop
scanRequests.stream()
  .filter(s -> s.getId().equals(id))
  .findFirst();

// Manual add
scanRequests.add(request);

// Manual remove
scanRequests.removeIf(
  s -> s.getId().equals(id));
```

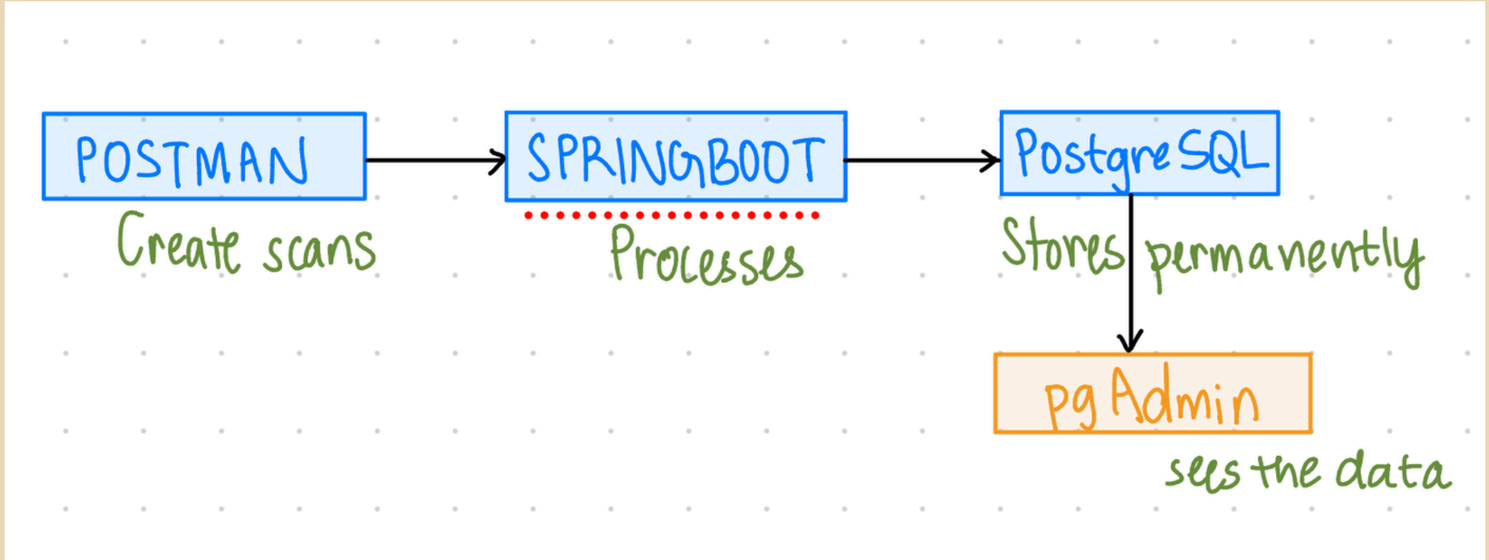
Stage 3:

```
// Database - survives restarts/crashes
@Autowired
private ScanRepository
  scanRepository;

// Hibernate generates: SELECT * WHERE id = ?
scanRepository.findById(id);

// Hibernate generates:
//INSERT INTO scan_requests VALUES (...)
scanRepository.save(request);

// Hibernate generates:
// DELETE FROM scan_requests WHERE id = ?
scanRepository.deleteById(id);
```

Databases solve our Concurrency Problem as well

Atomocity: Operation fully completes or fully fails	**Consitency:** Data always stays in a valid state
Isolation: Concurrent operations don’t interfere with each other	**Durability:** Once saved, survives crashes and restarts

Live Demo



Code

Enhancements

- Switch to MySQL or Oracle : one config change, zero Java changes
- Add Spring Security : authentication for API endpoints
- Docker : containerize for consistent deployment anywhere
- Multiple microservices : Patient Service, Scheduling Service communicating via REST
- API Gateway : single entry point routing to all services
- Cloud deployment : AWS, Azure, or GCP